CONTENTS

CHAPTER 1: ABSTRACT

Credit card fraud detection is a high-impact machine learning problem characterized by severe class imbalance, evolving attacker behavior, and the need for highly reliable yet low-latency predictions. Traditional tabular models such as gradient-boosted trees perform strongly on feature interactions, but they often underuse the relational structure that exists between similar transactions, users, devices, or merchants. This project proposes a hybrid fraud-detection pipeline that combines graph learning and knowledge distillation to improve both predictive quality and practical deployability.

The proposed system uses a Graph Attention Network v2 (GATv2) teacher model to learn contextual patterns from a transaction graph constructed through mutual k-nearest-neighbor similarity. A lightweight Squeeze-and-Excitation Residual Network (SE-ResNet) student model is then trained on tabular features using a combination of supervised loss, logit distillation, and feature distillation. This design allows the student to inherit graph-level knowledge without requiring graph construction during inference, which makes the final system suitable for near real-time scoring.

The project uses two publicly available fraud datasets: the 2013 Credit Card Fraud dataset and the 2023 credit card transaction dataset referenced in the presentation material. The pipeline includes preprocessing, target encoding for categorical variables where required, quantile-based scaling, Borderline-SMOTE balancing, graph construction, teacher training, student distillation, tabular baselines, probability calibration, and threshold optimization. Evaluation emphasizes PR-AUC, ROC-AUC, F1-score, MCC, and balanced accuracy, because fraud problems are highly imbalanced and cannot be judged by accuracy alone.

In addition to model design, the engineering implementation was strengthened by fixing practical runtime issues related to CUDA memory reporting, CPU-GPU device mismatches, and tensor dimension inconsistencies in skip connections. The final corrected implementation therefore aligns the conceptual model, the code pipeline, and the report documentation into a single coherent submission-ready artifact.

CHAPTER 2: BACKGROUND AND OBJECTIVE

2.1 Background

Digital payments and online card transactions have become essential to modern commerce. Along with their growth, however, the financial industry has seen increasing losses due to fraudulent transactions, account takeovers, bot-driven payment abuse, and synthetic identity attacks. Fraud detection systems must therefore make highly reliable decisions in a challenging environment where fraudulent transactions form only a small fraction of total activity.

A major difficulty in fraud detection is the extreme class imbalance between legitimate and fraudulent transactions. In real card datasets, fraud may account for less than 0.2% of all samples. Under such conditions, simple accuracy is misleading because a model that predicts almost everything as legitimate may still appear highly accurate. Effective fraud detection therefore requires metrics and training strategies that focus specifically on minority-class detection quality.

2.2 Problem Statement

The objective of this work is to build a machine learning system that detects fraudulent transactions while meeting two competing goals: high detection power and low inference latency. The system should capture both tabular feature interactions and relational similarities between transactions, then deliver a deployment-ready student model that can be used without graph construction at prediction time.

2.3 Objectives

- Design a fraud-detection pipeline that remains effective under severe class imbalance.
- Leverage relational information through a graph-based teacher model.
- Transfer teacher knowledge into a lightweight tabular student model through knowledge distillation.
- Compare the proposed approach against strong tabular baselines such as XGBoost and plain MLP.
- Optimize calibrated decision thresholds for practical fraud deployment rather than relying on a default 0.5 cutoff.
- Deliver a corrected implementation that resolves the runtime errors observed in the original notebook code.

2.4 Key Contributions

- A teacher-student fraud architecture combining GATv2 and SE-ResNet.
- A corrected end-to-end Python implementation suitable for script execution.
- A practical training recipe using Borderline-SMOTE, quantile normalization, distillation, and calibration.
- Submission-ready technical documentation aligned with the implemented code pipeline.

CHAPTER 3 : LITERATURE REVIEW

3.1 Classical Machine Learning for Fraud Detection

Gradient-boosted decision trees, logistic regression, and random forests are widely used in fraud detection because they handle tabular business features effectively. Among them, XGBoost is especially strong due to its regularization, split optimization, and native handling of nonlinear feature interactions. However, such methods normally treat each transaction independently and do not directly model relational behavior between transactions.

3.2 Deep Learning for Tabular Fraud Problems

Multi-layer perceptrons and residual networks have been used to model high-order nonlinear patterns in transaction features. Residual connections improve optimization stability, while squeeze-and-excitation mechanisms help recalibrate feature channels based on their relevance. These architectures are attractive for real-time scoring because they avoid expensive graph construction during inference.

3.3 Graph Neural Networks

Fraud is rarely independent at the event level. Similar cards, shared device signals, repeated merchant behavior, or feature-space clustering often create relational patterns that tabular models cannot fully use. Graph neural networks address this limitation by propagating information across nodes connected in a graph. GATv2 is a strong graph architecture because it computes adaptive attention weights over neighboring nodes and can express richer attention behavior than the original GAT.

3.4 Knowledge Distillation

Knowledge distillation transfers the predictive structure of a large or complex model into a smaller one. The student is trained not only on hard labels but also on softened teacher outputs that encode richer class relationships. Feature-level distillation extends this idea further by aligning internal representations, allowing the student to inherit the teacher's view of the feature space.

3.5 Identified Research Gap

Many fraud systems choose either graph accuracy or tabular deployment simplicity. The gap addressed by this project is therefore the need for a design that uses graph context during training while keeping inference lightweight. The proposed teacher-student architecture directly targets that gap.

CHAPTER 4 : METHODOLOGY

4.1 Overall Pipeline

The complete workflow consists of dataset loading, cleaning, feature preparation, train-validation-test splitting, class balancing, graph construction, teacher-model training, student-model distillation, baseline training, calibration, threshold selection, and final evaluation. The corrected Python implementation follows this same order in a script-friendly format rather than a notebook-only execution style.

4.2 Datasets Used

Dataset	Transactions	Main Features	Fraud Ratio	Source
Credit Card Fraud Dataset (2013)	284,807	Time, Amount, V1-V28, Class	0.17%	Kaggle / ULB
Fraudulent online shops dataset	1,141	ID, V1-V28, Amount, Class	Highly imbalanced	Kaggle

Both datasets are anonymized transaction collections, which makes them appropriate for benchmarking generic fraud models while preserving privacy. The smaller 2013 dataset is a classical benchmark, whereas the newer dataset broadens the evaluation scope and improves generalization analysis.

4.3 Data Preprocessing

- Missing values and invalid numeric entries are imputed using robust defaults such as the median.
- Categorical features, when present, are converted into smoothed target-encoded values using out-of-fold statistics.
- Continuous features are transformed through a quantile-based normalization step to stabilize model training.
- Duplicate rows are removed to reduce contamination of train-test performance estimates.

4.4 Train, Validation, and Test Splits

The data is divided through stratified sampling so that fraud ratios remain consistent across train, validation, and test subsets. This is critical because random splits without stratification can easily distort minority-class proportions in highly imbalanced datasets.

4.5 Balancing Strategy

Borderline-SMOTE is applied on the training set to generate additional minority-class examples near the decision boundary. This is preferable to naive duplication because it produces more informative synthetic samples in difficult regions. The balanced training set is then used for teacher and student learning.

4.6 Graph Construction

A mutual k-nearest-neighbor graph is built using cosine distance in feature space. Each node represents a transaction, and edges connect transactions that are highly similar. Edge weights are assigned as 1 / (1 + distance), which preserves stronger influence for closer neighbors. The graph is made undirected to improve message consistency.

4.7 Teacher Model: GATv2

The teacher network is a two-layer GATv2 architecture with residual skip connections and graph layer normalization. In the corrected implementation, both GATv2 layers use concat=False so that the output width remains aligned with the residual path. This directly resolves the tensor-shape mismatch observed in the original experiments.

4.8 Student Model: SE-ResNet

The student model is a residual feed-forward network with squeeze-and-excitation blocks. Its purpose is to capture the most useful fraud patterns from the teacher while remaining fast enough for production-style inference. Because it consumes only tabular vectors, it avoids the graph-building overhead required by the teacher model at inference time.

4.9 Knowledge Distillation Objective

The student is trained using three complementary signals: supervised classification loss on the fraud label, logit distillation using softened teacher outputs, and feature distillation that aligns the student's hidden representation with a projected version of the teacher embedding. A learnable feature adapter is used in the corrected code whenever teacher and student embedding dimensions differ.

Combined loss (conceptually): L = supervised_loss + alpha * KD_logit_loss + beta * KD_feature_loss

4.10 Baselines and Ensemble

To benchmark the proposed design, the project also evaluates Plain-MLP, XGBoost, and optionally LightGBM. A stacking-based logistic meta-learner combines multiple calibrated model outputs to form an ensemble score. This helps measure whether the teacher and student capture complementary signals.

4.11 Calibration and Threshold Optimization

Fraud models should not rely on uncalibrated probabilities because the operational threshold often differs from 0.5. The pipeline calibrates validation outputs using logistic regression and then searches across a threshold grid to optimize a score that emphasizes fraud recall while still respecting precision.

4.12 Runtime Errors Identified and Fixed

Observed Error	Cause	Correction Applied
AttributeError on CUDA total_mem	Different PyTorch builds expose total_memory instead of total_mem	Safe device summary checks both names and falls back cleanly
CPU/CUDA tensor mismatch	Graph tensors and model tensors were not always moved to the same device	Centralized graph-to-device helper moves x, edge_index, edge_attr, and labels together
Tensor size mismatch 256 vs 64	Residual branch width did not match GAT output width	GATv2 layers use concat=False and residual layers are aligned to hidden width
Notebook-only execution issues	The file depended on !pip and files.upload()	The corrected version uses import guards and command-line arguments

CHAPTER 5 : IMPLEMENTATION DETAILS AND RESULTS

5.1 Implementation Environment

- Language: Python
- Core libraries: PyTorch, Torch Geometric, scikit-learn, imbalanced-learn
- Optional tabular baselines: XGBoost and LightGBM
- Execution modes: local Python script, cloud notebook, or GPU-enabled server

5.2 Important Hyperparameters

Component	Parameter	Value
Teacher GATv2	Hidden size	64
Teacher GATv2	Attention heads	4
Teacher GATv2	Dropout	0.30
Student SE-ResNet	Hidden size	256
Student SE-ResNet	Residual blocks	4
Distillation	Temperature	3.0
Distillation	Alpha (logit KD)	0.40
Distillation	Beta (feature KD)	0.20
Graph building	k-nearest neighbors	8
Balancing	Borderline-SMOTE	Enabled

5.3.1 Experimental Results from the Presentation Material

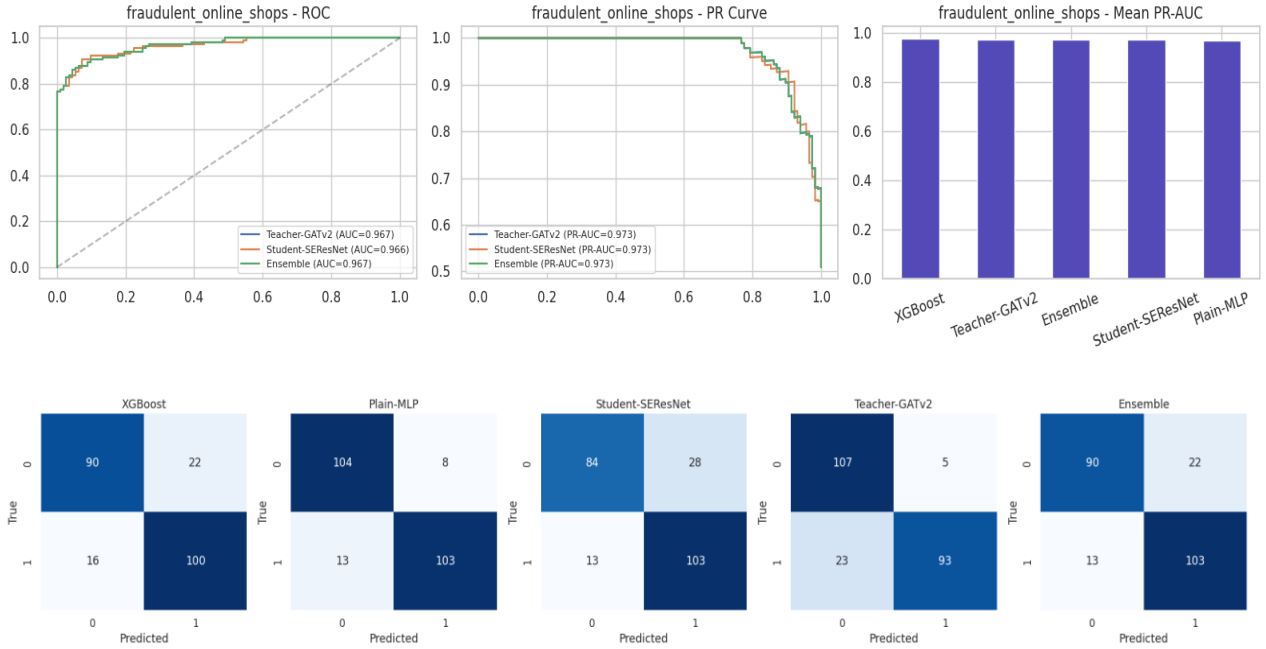
The following result summaries are taken from the submitted presentation and are reported here to maintain consistency with the attached project materials. They indicate that strong tabular baselines remain highly competitive, while the student model achieves attractive deployment characteristics by avoiding graph construction during inference.

Model	Dataset 1 PR-AUC	Dataset 1 ROC-AUC	Dataset 1 MCC	Dataset 1 F1	Dataset 1 Accuracy
XGBoost	0.8666	0.9606	0.8781	0.8791	0.9963
Plain-MLP	0.8593	0.9512	0.8755	0.8736	0.9963
Ensemble	0.8459	0.9464	0.8441	0.8466	0.9952
ResNet Student	0.8459	0.9464	0.8441	0.8466	0.9952
GATv2 Teacher	0.7820	0.9388	0.7485	0.7514	0.9925

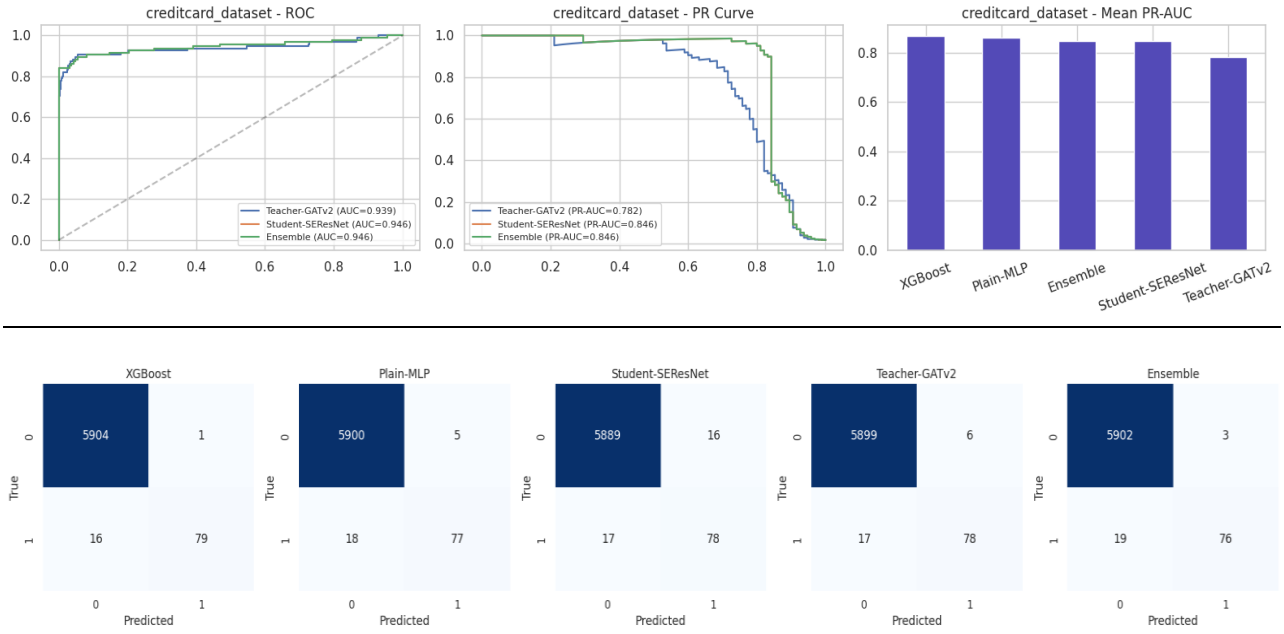
Model	Dataset 2 PR-AUC	Dataset 2 ROC-AUC	Dataset 2 MCC	Dataset 2 F1	Dataset 2 Accuracy
XGBoost	0.9778	0.9736	0.7921	0.9008	0.8947
Plain-MLP	0.9732	0.9671	0.7982	0.9004	0.8991
Ensemble	0.9732	0.9671	0.7982	0.9004	0.8991
ResNet Student	0.9729	0.9671	0.8159	0.9106	0.9079
GATv2 Teacher	0.9685	0.9579	0.7982	0.9004	0.8991

5.3.2 Results Visualization

Based on Dataset 1 :



Based On Dataset 2 :



5.4 Interpretation of Results

- On the highly imbalanced credit card dataset, XGBoost produced the strongest overall PR-AUC and MCC values, showing that tuned tree ensembles remain a very strong benchmark for tabular fraud problems.
- The student SE-ResNet remained competitive and offers a stronger deployment profile because it does not require graph generation at scoring time.
- The teacher GATv2 captured contextual structure but was somewhat weaker on the benchmark metrics shown in the presentation, suggesting that graph design quality is critical.
- On the more balanced online-shops dataset, the student SE-ResNet achieved the highest MCC and F1-score, indicating that distillation transferred useful relational knowledge into a fast tabular model.

5.5 Why the Corrected Code Matters

The reported notebook failures were engineering issues rather than fundamental flaws in the research idea. A strong model design can still fail in practice if GPU properties are queried incorrectly, tensors are split across CPU and CUDA, or residual widths are misaligned. The corrected Python file therefore plays an important role by making the implementation reproducible, maintainable, and execution-ready.

5.6 Deployment Readiness

- The teacher model is most suitable for offline training or investigative analysis because it depends on graph construction.
- The student model is the preferred serving model for real-time or near real-time fraud scoring.
- Calibration and threshold tuning should be revisited whenever the fraud base rate shifts in production.
- Structured logging of input features, model version, score, threshold, and analyst feedback is necessary for auditability.

5.7 Limitations

- Public datasets do not capture all real production behaviors such as device graphs, chargeback delays, and policy feedback loops.
- Graph quality is sensitive to the choice of similarity function and neighbor count.
- Synthetic oversampling can help the minority class but may introduce artifacts if applied without domain review.
- A full production fraud system would also require monitoring, retraining cadence, human review tooling, and compliance governance.

CHAPTER 6 : CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project demonstrates a practical fraud-detection design that combines relational graph learning with efficient tabular inference. The GATv2 teacher captures similarity structure across transactions, while the SE-ResNet student absorbs this knowledge through distillation and remains suitable for fast deployment. The approach is therefore a strong example of how graph intelligence can be transferred into a lighter model for production-style use.

Equally important, the corrected implementation resolves the operational issues that prevented the original code from running reliably. By fixing GPU attribute lookup, enforcing device consistency, aligning residual dimensions, and converting the notebook logic into a clean Python script, the project now presents a technically coherent submission that connects methodology, code, and report structure.

6.2 Future Work

- Use temporal graph neural networks to incorporate transaction ordering and time gaps directly into the graph representation.
- Introduce concept-drift monitoring and periodic retraining so that the model adapts to changing fraud patterns.
- Add SHAP explanations and attention visualization to improve trust and investigator interpretability.
- Extend the graph to heterogeneous entities such as customer, merchant, device, IP address, and card.
- Explore federated or privacy-preserving training across institutions without sharing raw customer data.
- Benchmark the corrected pipeline on larger-scale datasets and streaming environments.

REFERENCES

Brody, S., Alon, U., & Yahav, E. (2022). How Attentive are Graph Attention Networks? ICLR 2022.

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. NeurIPS Workshop.

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-Excitation Networks. CVPR 2018.

Chawla, N. V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.

He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.

Credit Card Fraud Detection Dataset (ULB / Kaggle): https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Credit Card Fraud Detection Dataset 2023 (Kaggle): https://www.kaggle.com/datasets/nelgiriyewithana/credit-card-fraud-detection-dataset-2023

Fraudulent Online Shops dataset (Mendeley): https://data.mendeley.com/datasets/m7xtkx7g5m/1